



INSTITUTO SUPERIOR TECNOLÓGICO QUITO

Carrera de Desarrollo de Software

Asignatura: Ambientes Computacionales

TRABAJO FINAL DE ASIGNATURA

TIMEBANK

Aplicación web de banco de tiempo para el intercambio de servicios
por horas

Segundo avance

Integrantes:

Kevin Adrián Quiñonez Rocha

Orlando Eloy Trujillo

Docente: Ing. Eldis Gabriel Nicolalde Navarrete

Nivel: Cuarto

Quito, octubre de 2026



Nombre del Estudiante	Kevin Adrián Quiñonez Rocha y Orlando Eloy Trujillo	Fecha de entrega:	07/10/2026
Asignatura:	Ambientes Computacionales		
Carrera:	Desarrollo de Software	Nivel:	Cuarto
Nombre del Docente:	Ing. Eldis Gabriel Nicolalde Navarrete		

TRABAJO FINAL DE ASIGNATURA

TIMEBANK: APLICACIÓN WEB DE BANCO DE TIEMPO PARA EL INTERCAMBIO DE SERVICIOS POR HORAS

Segundo avance · corte al 7 de octubre de 2026

1. Descripción del trabajo final

En muchas comunidades —barrios, instituciones educativas o grupos de vecinos— hay personas que saben hacer algo útil, como dar clases, reparar equipos o cuidar a niños y adultos mayores, pero que no siempre tienen dinero para pagar por la ayuda que necesitan. Los bancos de tiempo proponen usar la hora como unidad de intercambio: quien dedica una hora a ayudar a otra persona recibe una hora que luego puede usar para pedir un servicio, sin importar de qué servicio se trate (Cahn, 2000).

TimeBank es una aplicación web que lleva ese registro de forma ordenada y segura. Los usuarios publican los servicios que ofrecen dentro de una categoría, solicitan los servicios de otras personas, aceptan o rechazan las solicitudes que reciben y, cuando un intercambio se completa, el sistema transfiere las horas y guarda el movimiento. Cada usuario nuevo recibe dos horas de bienvenida y puede valorar de 1 a 5 los servicios recibidos.

El proyecto se desarrolla en pareja con una API REST en ASP.NET Core, una base de datos PostgreSQL y un frontend en Angular, con el código en un repositorio público de GitHub. Este documento presenta el avance hasta el segundo corte: estructura del proyecto, base de datos, CRUD de la API, autenticación con JWT, roles de usuario y frontend conectado con inicio de sesión.

2. Objetivo General

Integrar una aplicación web de banco de tiempo mediante una API REST en ASP.NET Core, una base de datos PostgreSQL y un frontend en Angular protegido con autenticación JWT y roles, para que los miembros de una comunidad registren e intercambien servicios usando horas como unidad de pago.



3. Objetivos Específicos

- Determinar los requerimientos funcionales, las entidades y las reglas de negocio de un banco de tiempo, considerando el registro de usuarios, servicios, solicitudes, valoraciones y el saldo de horas.
- Establecer la API REST con Entity Framework Core y PostgreSQL, protegida con autenticación JWT y autorización por roles, y conectarla con el frontend desarrollado en Angular.
- Verificar el funcionamiento del sistema mediante pruebas de los endpoints en Postman, scripts automatizados y pruebas de la interfaz en el navegador.

4. Resultado de Aprendizaje

[Copiar aquí el resultado de aprendizaje del PEA de Ambientes Computacionales]

5. Recursos necesarios

Tabla 1. Recursos utilizados en el proyecto.

Tipo	Recurso	Uso en el proyecto
Hardware	Computador portátil con Windows 11	Desarrollo, ejecución local y pruebas (localhost)
Software	Visual Studio Code	Editor de código y terminal integrada
Software	.NET 10 SDK · ASP.NET Core 10 Web API (C#)	Backend: controladores, servicios y seguridad
Software	Entity Framework Core 10 · Npgsql 10.0.3 · dotnet-ef	Acceso a datos y migraciones
Software	PostgreSQL y pgAdmin 4	Base de datos y verificación de tablas
Software	Node.js, npm y Angular 22 (CLI)	Frontend de la aplicación
Software	Microsoft.AspNetCore.Authentication.JwtBearer 10.0.12	Validación de los tokens JWT
Software	Postman y PowerShell	Pruebas manuales y automatizadas de la API
Software	Git y GitHub	Control de versiones y repositorio público
Software	Navegador web con herramientas de desarrollador (F12)	Pruebas de la interfaz y revisión del token
Humanos	Dos estudiantes y el docente de la asignatura	Desarrollo, revisión y retroalimentación

6. Desarrollo

El trabajo se organizó en fases que siguen el orden de las clases de la asignatura. Cada fase terminó con un commit en GitHub, de modo que el historial del repositorio muestra la evolución del proyecto.

6.1. Fase 1: Control de versiones y estructura del proyecto

Git es un sistema de control de versiones distribuido que guarda el historial de cambios, registra quién hizo cada cambio y permite volver a versiones anteriores (Chacon y Straub, 2014). Se creó un repositorio público en GitHub y se definió una estructura que separa cada parte del sistema:

- frontend/timebank-web: aplicación Angular.
- backend/TimeBank.API: API en ASP.NET Core, organizada en Controllers, Services, Dtos, Models, Data, Security, Extensions y Migrations.
- database, docs y tests: scripts, documentación, colección de Postman, capturas, presentaciones y pruebas automáticas.

Se aplicó la práctica de commits coherentes: cada commit agrupa un cambio con lógica completa (por ejemplo, todo el módulo de login) y cada avance se marcó con una etiqueta de versión.

Tabla 2. Historial de versiones del proyecto en GitHub.

Fecha	Commit / versión	Contenido
25/09/2026	2798630 · 9dc8bb3	Estructura inicial y carpetas del proyecto
28/09/2026	d6a07b9 · v2.0	Modelos y conexión a PostgreSQL
29/09/2026	985410f · v3.0	CRUD de la API con reglas de negocio y pruebas (primer avance)
01/10/2026	ca0889a · v4.0	Login con JWT y sesiones
06/10/2026	c185dfd	Roles, contraseña segura y CRUD protegido con token
06/10/2026	488a6cb	Frontend Angular conectado a la API (Actividad 4.3)
07/10/2026	a392148	Login en Angular y rutas por rol (Actividad 5.3)

6.2. Fase 2: Análisis y modelo de datos

A partir de la idea del banco de tiempo se identificaron las entidades del sistema y sus relaciones. Las clases se escribieron en C# siguiendo las convenciones de Entity Framework Core: propiedades públicas, textos inicializados con string.Empty, campos opcionales con string? y relaciones de uno a muchos con ICollection<T>.

Tabla 3. Entidades del modelo de datos de TimeBank.

Entidad	Qué representa	Relaciones principales
User	Persona registrada: nombre, correo único, contraseña con hash y saldo de horas	Ofrece servicios, hace solicitudes, tiene roles y sesiones
Category	Tipo de servicio (tutorías, tecnología, hogar...) con nombre único	Agrupar muchos servicios

Entidad	Qué representa	Relaciones principales
Service	Servicio que ofrece un usuario	Pertenece a un usuario y a una categoría
ServiceRequest	Pedido de un servicio con su estado	Une al solicitante con el servicio
Review	Valoración de 1 a 5 de un intercambio completado	Pertenece a una solicitud
TimeTransaction	Movimiento de horas entre dos usuarios	Se genera al completar una solicitud
Role / UserRole	Roles Administrador y Usuario	Relación de muchos a muchos con User
UserSession	Sesión iniciada: token (jti), fecha, IP y estado	Pertenece a un usuario

Las reglas de negocio definidas fueron:

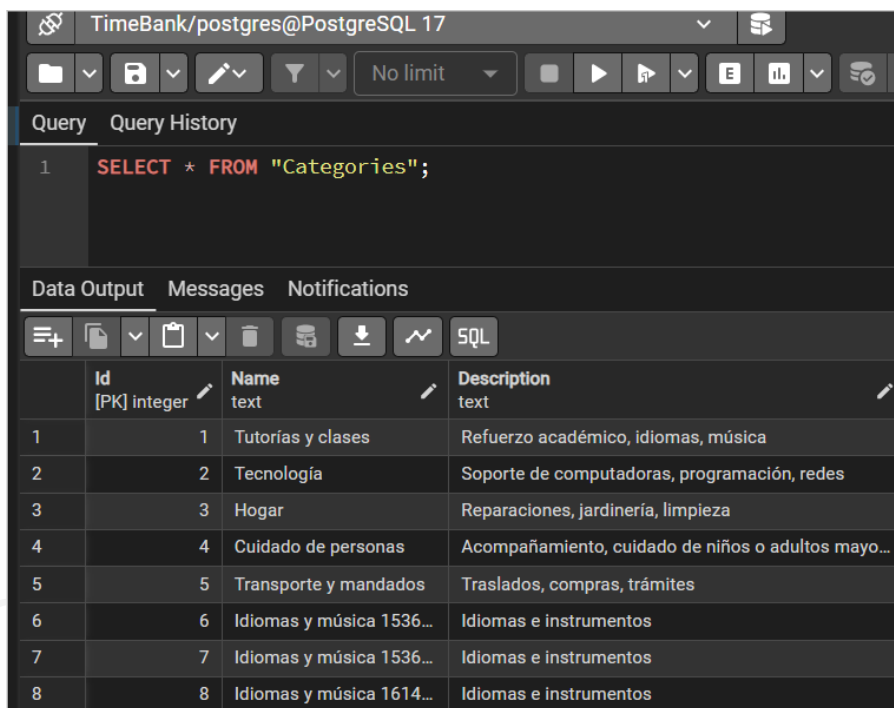
- Cada usuario nuevo recibe **2 horas de bienvenida**.
- El correo de cada usuario y el nombre de cada categoría son únicos.
- Nadie puede pedir su propio servicio y, al solicitar, se valida que el saldo de horas sea suficiente.
- Una solicitud pasa de **Pendiente** a **Aceptada** o **Rechazada**, y de Aceptada a **Completada**. Al completarse se transfieren las horas y se registra la TimeTransaction.
- Las valoraciones van de 1 a 5 y solo se permiten cuando el intercambio está completado.
- Si un registro tiene historial, se desactiva en lugar de borrarse.

6.3. Fase 3: Conexión a PostgreSQL y migraciones

Inicialmente se planteó SQL Server, pero por problemas para descargarlo se usó **PostgreSQL** con pgAdmin 4. Se instalaron los paquetes Npgsql.EntityFrameworkCore.PostgreSQL y Microsoft.EntityFrameworkCore.Design, y la herramienta dotnet-ef. La cadena de conexión se guardó en appsettings.Development.json, archivo que se excluyó del repositorio para no publicar la contraseña de la base de datos.

En TimeBankDbContext cada DbSet<T> se convierte en una tabla. Con las migraciones de Entity Framework Core el esquema de la base de datos se genera y actualiza a partir de las clases (Microsoft, s. f.-a):

```
dotnet ef migrations add InitialCreate # modelos del primer avance
dotnet ef migrations add AddUserSessions # sesiones del login
dotnet ef migrations add AddRoles # roles de usuario
dotnet ef database update
```



	Id [PK] integer	Name text	Description text
1	1	Tutorías y clases	Refuerzo académico, idiomas, música
2	2	Tecnología	Soporte de computadoras, programación, redes
3	3	Hogar	Reparaciones, jardinería, limpieza
4	4	Cuidado de personas	Acompañamiento, cuidado de niños o adultos mayo...
5	5	Transporte y mandados	Traslados, compras, trámites
6	6	Idiomas y música 1536...	Idiomas e instrumentos
7	7	Idiomas y música 1536...	Idiomas e instrumentos
8	8	Idiomas y música 1614...	Idiomas e instrumentos

Figura 1. Tabla *Categories* consultada en pgAdmin con `SELECT * FROM "Categories"`.

6.4. Fase 4: API REST y operaciones CRUD

La API sigue una arquitectura por capas: el **controlador** recibe la petición HTTP en formato JSON, el **servicio** aplica las reglas de negocio y **Entity Framework Core** traduce las operaciones a SQL para PostgreSQL. El DbContext y los servicios se reciben por inyección de dependencias en el constructor.

Tabla 4. Endpoints del CRUD de TimeBank.

Recurso	Ruta	Operaciones
Categorías	/api/categories	Listar, ver por id, crear, editar y eliminar
Usuarios	/api/users	Listar y buscar, ver por id, crear, editar, cambiar roles y eliminar
Servicios	/api/services	Listar con filtros por categoría y texto, ver, crear, editar y eliminar
Solicitudes	/api/servicerequests	CRUD y cambio de estado: aceptar, rechazar y completar (PUT /{id}/status)
Valoraciones	/api/reviews	Listar, ver por id, crear y eliminar
Movimientos	/api/timetransactions	Historial de horas

La API responde con los códigos HTTP correspondientes: **200** (correcto), **201** (creado), **204** (actualizado o eliminado, sin contenido), **400** (datos inválidos), **401** (sin token), **403** (sin permiso), **404** (no encontrado) y **409** (correo ya registrado). Los mensajes de error se devuelven en español y sin detalles técnicos.

```
PS C:\Users\kadri\Desktop\ITQ\TimeBank> cd backend\TimeBank.API
PS C:\Users\kadri\Desktop\ITQ\TimeBank\backend\TimeBank.API> dotnet run
Usando la configuración de inicio de C:\Users\kadri\Desktop\ITQ\TimeBank\backend\TimeBank.API\Properties\l
aunchSettings.json...
Compilando...
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5070
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: C:\Users\kadri\Desktop\ITQ\TimeBank\backend\TimeBank.API
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (43ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT c."Id", c."Description", c."Name"
      FROM "Categories" AS c
      ORDER BY c."Name"
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (3ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT c."Id", c."Description", c."Name"
      FROM "Categories" AS c
      ORDER BY c."Name"
```

Figura 2. API de TimeBank ejecutándose con dotnet run en <http://localhost:5070>.

6.5. Fase 5: Autenticación con JWT y sesiones

Para el inicio de sesión se usó **JSON Web Token (JWT)**, un formato estándar para transmitir de forma compacta y firmada la identidad de un usuario entre dos partes (Jones et al., 2015). El flujo es el siguiente:

- Angular envía el correo y la contraseña a `POST /api/auth/login`.
- La API busca al usuario y verifica la contraseña contra su hash con `PasswordHasher`. Si falla el correo o la contraseña, responde el mismo mensaje con código 401, para no dar pistas a un atacante.
- Si los datos son correctos, genera un token firmado con `HMAC-SHA256` que dura **120 minutos** e incluye el id del usuario (sub), su correo, su nombre, un identificador único (jti) y sus roles.
- La sesión se guarda en la tabla `UserSessions` con la fecha, la IP y el navegador. En cada petición, el evento `OnTokenValidated` comprueba que la sesión siga activa; al cerrar sesión se revoca el jti y el token deja de servir aunque no haya vencido.

Las contraseñas nunca se guardan en texto plano: se almacenan con hash, como recomienda la guía de OWASP (OWASP Foundation, s. f.).

Tabla 5. Endpoints del módulo de autenticación.

Método	Ruta	Función
POST	/api/auth/register	Registrar una cuenta con el rol Usuario
POST	/api/auth/login	Iniciar sesión y recibir el token y el perfil
GET / PUT	/api/auth/me	Ver o editar el perfil propio
PUT	/api/auth/change-password	Cambiar la contraseña y cerrar las otras sesiones
GET	/api/auth/sessions	Listar las sesiones del usuario
DELETE	/api/auth/logout	Cerrar la sesión actual

Método	Ruta	Función
DELETE	/api/auth/sessions/{id}	Cerrar una sesión específica

6.6. Fase 6: Contraseña segura y autorización por roles

Se creó el atributo de validación [StrongPassword], que exige entre 8 y 64 caracteres, al menos una mayúscula, una minúscula, un número y un carácter especial, y que no haya espacios (por ejemplo, Clave2026*). Si la contraseña no cumple, la API responde 400 con un mensaje claro.

Se definieron dos roles: **Administrador** y **Usuario**. Quien se registra queda como Usuario y solo el Administrador puede asignar roles privilegiados desde `PUT /api/users/{id}/roles`. Todos los controladores heredan de `ApiControllerBase`, que tiene el atributo [Authorize]; las acciones exclusivas usan [Authorize(Roles = AppRoles.Admin)] (Microsoft, s. f.-b). Así, una petición sin token responde **401** y una petición con token pero sin permiso responde **403**.

Tabla 6. Permisos por rol en la API.

Recurso	Administrador	Usuario
Categorías	Ver, crear, editar y eliminar	Ver
Usuarios y roles	Gestión completa y asignación de roles	Ver un usuario por id
Servicios	Gestionar todos	Ver todos; editar y eliminar solo los propios
Solicitudes	Gestionar todas	Crear, editar y eliminar las propias; aceptar o rechazar si ofrece el servicio; completar si lo pidió
Valoraciones	Gestionar todas	Crear y eliminar las que él mismo hizo
Movimientos de horas	Ver todos	Ver solo los propios

6.7. Fase 7: Conexión del frontend con la API

En Angular se habilitó el cliente HTTP con `provideHttpClient(withFetch())` en `app.config.ts` y se creó el servicio `TimeBankApiService`, que centraliza la URL base de la API (`http://localhost:5070/api`). El componente consulta las categorías al iniciar con `ngOnInit()` y `subscribe({ next, error })`, y la plantilla las muestra con `@if` y `@for` (Angular, s. f.-a).

En el backend, la política CORS **Frontend** permite peticiones solo desde `http://localhost:4200` (Microsoft, s. f.-c). El listado de categorías se marcó con [AllowAnonymous] para que cualquier visitante vea qué tipos de servicio ofrece la plataforma; crear, editar o eliminar categorías sigue siendo exclusivo del Administrador.

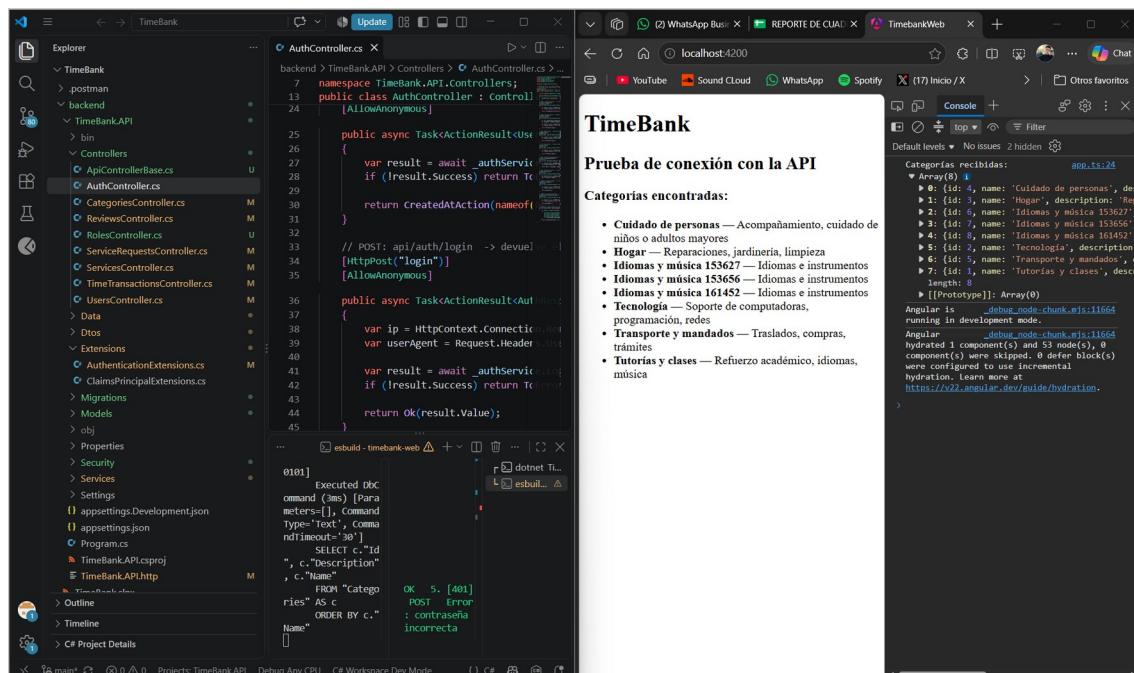


Figura 3. Base de datos, backend y frontend ejecutándose al mismo tiempo y conectados.

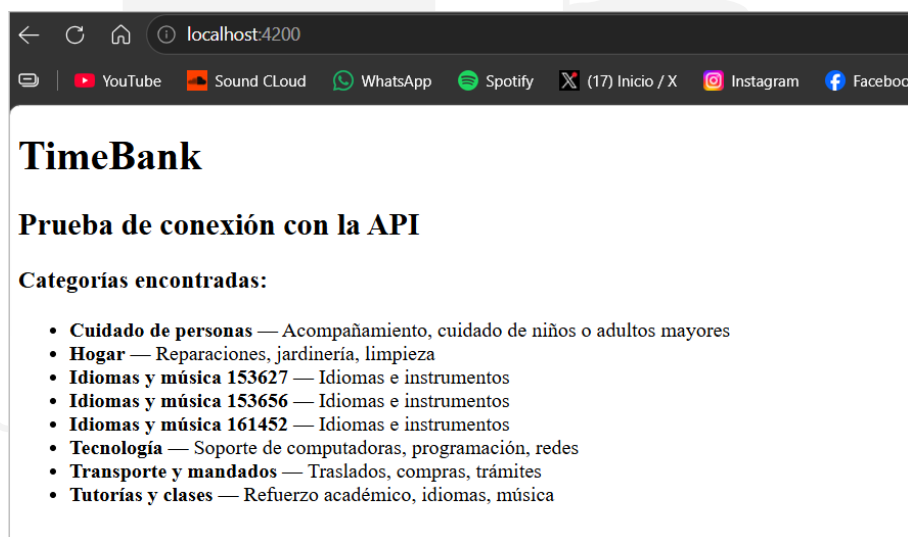


Figura 4. Página de TimeBank mostrando las categorías obtenidas de la API.

6.8. Fase 8: Inicio de sesión en Angular

Para completar la conexión se programó el login en el frontend con las siguientes piezas:

- **AuthService:** llama a `POST /api/auth/login`, guarda el token, su vencimiento y el usuario en `localStorage`, consulta el perfil con `GET /api/auth/me` y cierra la sesión con `DELETE /api/auth/logout`.
- **Interceptor:** agrega el encabezado `Authorization: Bearer <token>` a cada petición y, si la API responde 401, borra la sesión y envía al usuario al login (Angular, s. f.-b).

- **Guards:** authGuard exige sesión iniciada, guestGuard evita volver al login con sesión abierta y roleGuard solo deja entrar a /usuarios al rol Administrador (Angular, s. f.-c).
- **Pantallas:** /login con validaciones y mensajes claros, /inicio con el nombre, el rol, el saldo de horas y las categorías, y /usuarios con la lista de usuarios para el Administrador. El menú cambia según el rol.

Como el proyecto se creó con renderizado en el servidor (SSR) y localStorage solo existe en el navegador, las rutas se configuraron con `RenderMode.Client` para que se dibujen en el cliente.

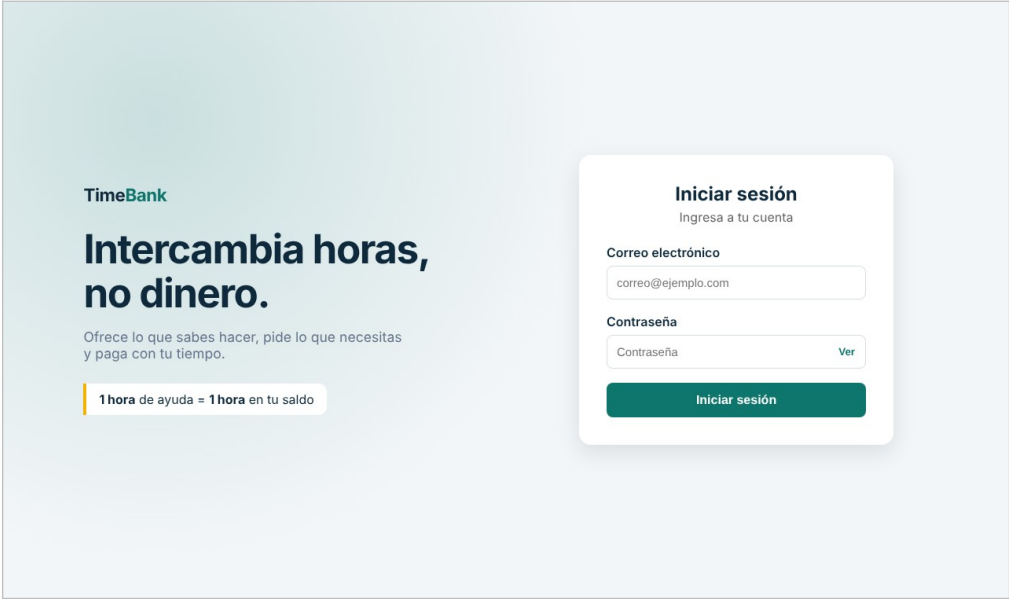


Figura 5. Pantalla de inicio de sesión de TimeBank.

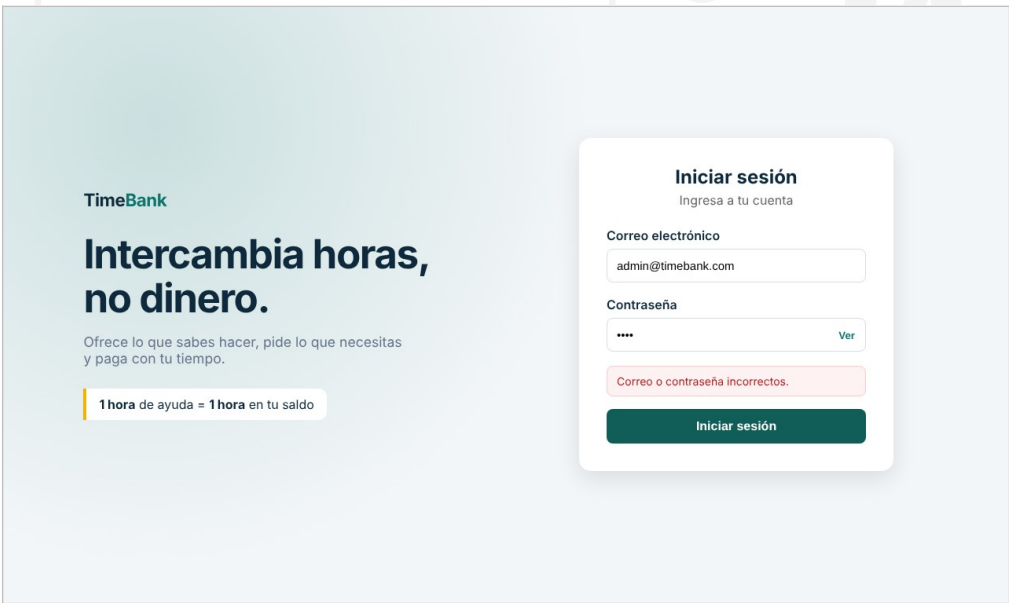


Figura 6. Mensaje mostrado cuando la contraseña es incorrecta (respuesta 401 de la API).

6.9. Fase 9: Pruebas y resultados

La API se probó con una colección de Postman organizada por módulos (docs/TimeBank API) y con dos scripts de PowerShell que recorren los casos correctos y de error: tests/probar-api.ps1 para el CRUD y tests/probar-login.ps1 para el registro, el inicio de sesión, las sesiones y el cierre de sesión. El frontend se probó en el navegador.

Tabla 7. Resumen de pruebas realizadas.

Prueba	Resultado esperado	Resultado
Listar usuarios sin token	401 No autorizado	Correcto
Crear categoría con token de Usuario	403 Prohibido	Correcto
Crear categoría con token de Administrador	201 Creado	Correcto
Registrar una contraseña débil	400 con mensaje de contraseña no válida	Correcto
Registrar un correo repetido	409 Conflicto	Correcto
Pedir un servicio propio o sin saldo	400 con mensaje de la regla de negocio	Correcto
Usar un token después del logout	401 (sesión revocada)	Correcto
Login en Angular con contraseña incorrecta	Mensaje "Correo o contraseña incorrectos"	Correcto
Login en Angular con datos correctos	Entra a /inicio con nombre, rol y saldo	Correcto
Usuario intenta abrir /usuarios	Vuelve a /inicio (roleGuard)	Correcto

6.10. Análisis de resultados y estado del avance

Al segundo corte, el backend cumple con los requisitos planteados por el docente: el login usa JWT, las contraseñas se guardan con hash y deben ser seguras, los roles privilegiados solo los asigna el Administrador y los errores se muestran con mensajes comprensibles para el usuario. Un aprendizaje importante fue la diferencia entre **autenticación** (saber quién es el usuario, 401) y **autorización** (saber qué puede hacer, 403).

En el frontend, la aplicación ya se comunica con la API, inicia y cierra sesión y adapta el menú según el rol. Quedan para el tercer avance las pantallas de gestión de servicios, solicitudes y valoraciones, el registro desde Angular, la confirmación antes de eliminar y el diseño adaptable a celulares.

Tabla 8. Estado de los componentes del proyecto.

Componente	Estado al segundo avance
Estructura, repositorio y versiones	Completado
Modelo de datos y migraciones en PostgreSQL	Completado

Componente	Estado al segundo avance
CRUD de la API con reglas de negocio	Completado
Login con JWT, sesiones y contraseña segura	Completado
Roles Administrador y Usuario en la API	Completado
Frontend conectado y login en Angular	Completado
Pantallas de servicios, solicitudes y valoraciones	Pendiente (tercer avance)
Registro de usuarios desde Angular	Pendiente (tercer avance)

7. Conclusiones:

- h. Se determinaron los requerimientos y las reglas de negocio del banco de tiempo y se tradujeron en nueve entidades relacionadas. Definir desde el inicio reglas como el saldo de bienvenida, los estados de una solicitud y la transferencia de horas permitió que la base de datos y la API respondan al problema real que se quiere resolver.
- i. Se estableció una API REST en ASP.NET Core con Entity Framework Core y PostgreSQL, protegida con JWT, sesiones revocables y roles, y conectada con un frontend en Angular. Separar el proyecto en capas y usar un interceptor y guards en Angular hizo que la seguridad se aplique de forma consistente en todo el sistema, sin repetir código en cada pantalla.
- j. Se verificó el funcionamiento del sistema con Postman, scripts automatizados y pruebas en el navegador, comprobando tanto los casos correctos como los de error (401, 403, 400 y 409). Esta práctica fortalece la formación profesional, porque acostumbra a probar primero el backend y a documentar cada cambio en un repositorio con historial.

8. Bibliografía:

- Angular. (s. f.-a). *Making HTTP requests*. Angular.
<https://angular.dev/guide/http/making-requests>
- Angular. (s. f.-b). *Interceptors*. Angular.
<https://angular.dev/guide/http/interceptors>
- Angular. (s. f.-c). *Control route access with guards*. Angular.
<https://angular.dev/guide/routing/route-guards>
- Cahn, E. S. (2000). *No more throw-away people: The co-production imperative*. Essential Books.
- Chacon, S. y Straub, B. (2014). *Pro Git* (2.^a ed.). Apress.
<https://git-scm.com/book/es/v2>
- Jones, M., Bradley, J. y Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force.
<https://doi.org/10.17487/RFC7519>



Microsoft. (s. f.-a). *Migrations overview*. Microsoft Learn.

<https://learn.microsoft.com/ef/core/managing-schemas/migrations/>

Microsoft. (s. f.-b). *Role-based authorization in ASP.NET Core*. Microsoft Learn.

<https://learn.microsoft.com/aspnet/core/security/authorization/roles>

Microsoft. (s. f.-c). *Enable Cross-Origin Requests (CORS) in ASP.NET Core*.

Microsoft Learn. <https://learn.microsoft.com/aspnet/core/security/cors>

OWASP Foundation. (s. f.). *Password storage cheat sheet*. OWASP Cheat Sheet Series.

https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

The PostgreSQL Global Development Group. (s. f.). *PostgreSQL documentation*. <https://www.postgresql.org/docs/>

9. Anexos:

- Repositorio del proyecto: <https://github.com/edrianlxcid/TimeBank>
- Presentaciones de los avances:
docs/presentaciones/TimeBank_Primer_Avance.pptx y
docs/presentaciones/TimeBank_Segundo_Avance.pptx.
- Colección de Postman: docs/TimeBank API. Guía del login:
docs/LOGIN.md.
- Evidencias de las actividades: docs/capturas/actividad-4.3 y
docs/capturas/actividad-5.3.

```
PS C:\Users\kadri\Desktop\ITQ\TimeBank\frontend\timebank-web> npm start
PS C:\Users\kadri\Desktop\ITQ\TimeBank\frontend\timebank-web> npm start

> timebank-web@0.0.0 start
> ng serve

Browser bundles
Initial chunk files | Names | Raw size
main.js            | main | 8.36 kB
styles.css         | styles | 95 bytes
Initial total      | 8.46 kB

Server bundles
Initial chunk files | Names | Raw size
main.server.mjs     | main.server | 9.58 kB
server.mjs          | server | 1.86 kB
polyfills.server.mjs | polyfills.server | 268 bytes

Application bundle generation complete. [13.243 seconds] - 2026-10-06T21:31:43.868Z

Watch mode enabled. Watching for file changes...
NOTE: Raw file sizes do not reflect development server per-request transformations.
→ Local: http://localhost:4200/
```

Figura 7. Frontend ejecutándose con `npm start` en `http://localhost:4200`.

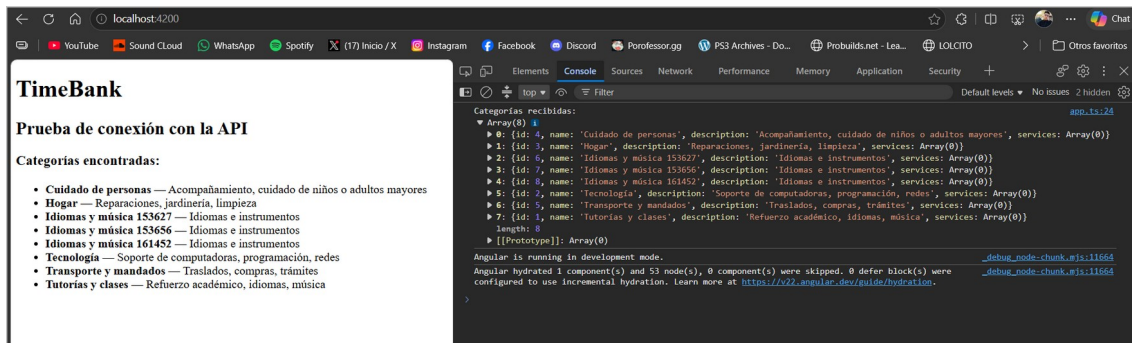


Figura 8. Consola del navegador con las categorías recibidas desde la API.